DBT Day 3

Shivam Zanjurne · Sat, 10 Oct 26 · Constraints, keys, foreign keys + referential actions, aggregates, GROUP BY / HAVING, LIKE, LIMIT / OFFSET, query execution order

---

Table of Contents

1. Constraints: Definition and Types
2. NULL Behavior in UNIQUE Columns
3. Constraint Demos: Success and Failures
4. Column-Level vs Table-Level Constraints
5. ALTER TABLE: Adding and Dropping Constraints
6. Data Integrity on UPDATE
7. Keys: Super, Candidate, Primary, Alternate
8. Composite and Surrogate Keys
9. Foreign Keys and Relationships
10. ON DELETE / ON UPDATE Actions
11. Aggregate Functions
12. GROUP BY, HAVING, ORDER BY, DISTINCT
13. LIMIT, OFFSET, BETWEEN, IN, LIKE, IS NULL
14. Logical Query Execution Order
15. Revision Checklist and Next Steps

Constraints: Definition and Types

What is a constraint?

A **constraint** is a **rule** attached to a table or column that controls what data may be inserted or updated. The database engine **rejects** statements that break the rule — you get an error instead of bad data.

Without constraints, any value can enter the table: duplicate emails, age = −5, NULL where an id is required. Constraints are how RDBMS enforces **data integrity** automatically.

Why they matter (extra explanation)

Without constraint	What can go wrong
No UNIQUE on email	Two users share one login email → identity chaos
No NOT NULL on id	Rows with “no identity” → hard to update/delete safely
No PRIMARY KEY	No stable row identity; joins become unreliable
No DEFAULT	Every insert must repeat the same boilerplate value
No CHECK on age	Age 5 or 200 slips in → business rules violated

Five constraints covered today

Constraint	Rule	NULLs?	How many per table
UNIQUE	No duplicate values in the column (or column set)	Multiple NULLs **allowed**	Many UNIQUE constraints OK
NOT NULL	Column must always have a value	NULLs **blocked**	Per column
PRIMARY KEY	UNIQUE + NOT NULL; main row identity	NULLs **blocked**	**Exactly one** (may be composite)
DEFAULT	If INSERT omits the column, use this value	N/A (fills a value)	Per column
CHECK	Value must satisfy a boolean condition	NULL usually passes CHECK in MySQL (unknown)	Many OK

Demo table with all five

```sql
CREATE TABLE constraint_demo (
    id         INT NOT NULL,
    name       VARCHAR(20) UNIQUE,
    email_id   VARCHAR(30) UNIQUE,
    age        INT CHECK (age > 18),
    department VARCHAR(20) DEFAULT 'IT'
);
```

Column	Constraint(s)
`id`	`NOT NULL`
`name`	`UNIQUE`
`email_id`	`UNIQUE`
`age`	`CHECK (age > 18)` — 18 itself fails; 19+ OK
`department`	`DEFAULT 'IT'` if omitted from INSERT

> **Note on CHECK:** `age > 18` means age must be **strictly greater** than 18. Age **18 fails**; age **19 succeeds**. If the business rule is "18 or older", write `CHECK (age ≥ 18)`.

NULL Behavior in UNIQUE Columns

Reminder: what NULL means

NULL is **not** `0`, not `''`, not "empty known value". It means **unknown / not provided**.

In SQL's three-valued logic, comparisons with NULL yield **UNKNOWN**, not TRUE:

```
NULL = NULL      -- not TRUE
NULL <> NULL     -- not TRUE
```

Why UNIQUE allows multiple NULLs

UNIQUE rejects a new value only when it finds an **existing equal value**. Because `NULL = NULL` is not TRUE, the engine does **not** treat two NULLs as duplicates.

```
-- First row: name omitted → NULL (UNIQUE column)
INSERT INTO constraint_demo (id, email_id, age, department)
VALUES (6, 'null1@gmail.com', 22, 'IT');

-- Second row: name also NULL — also succeeds
INSERT INTO constraint_demo (id, email_id, age, department)
VALUES (7, 'null2@gmail.com', 25, 'IT');
```

Both succeed: **NULL ≠ NULL** for uniqueness checking.

Constraint	Multiple NULLs?
UNIQUE	Allowed
NOT NULL	Blocked entirely
PRIMARY KEY	Blocked (PK implies NOT NULL)

> If you need "unique when present, but optional", UNIQUE alone is enough. If you need "must be present and unique", use **PRIMARY KEY** or **UNIQUE + NOT NULL**.

Constraint Demos: Success and Failures

Successful insert

```sql
INSERT INTO constraint_demo (id, name, email_id, age, department)
VALUES (1, 'Aniket', 'aniket@gmail.com', 20, 'Mechanical');
-- SUCCESS
```

UNIQUE violated on `name`

```sql
INSERT INTO constraint_demo (id, name, email_id, age, department)
VALUES (2, 'Aniket', 'takmare@gmail.com', 22, 'IT');
```

```
ERROR: Duplicate entry 'Aniket' for key 'constraint_demo.name'
```

UNIQUE violated on `email_id`

```sql
INSERT INTO constraint_demo (id, name, email_id, age, department)
VALUES (3, 'Takmare', 'aniket@gmail.com', 22, 'IT');
```

```
ERROR: Duplicate entry 'aniket@gmail.com' for key 'constraint_demo.email_id'
```

NOT NULL violated on `id`

```sql
INSERT INTO constraint_demo (id, name, email_id, age, department)
VALUES (NULL, 'Akshay', 'akshay@gmail.com', 21, 'IT');
```

```
ERROR: Column 'id' cannot be null
```

CHECK violated (`age = 18` , need `> 18`)

```sql
INSERT INTO constraint_demo (id, name, email_id, age, department)
VALUES (3, 'Akshay', 'akshay@gmail.com', 18, 'IT');
```

```
ERROR: Check constraint 'constraint_demo_chk_1' is violated
```

CHECK satisfied (`age = 19`)

```
INSERT INTO constraint_demo (id, name, email_id, age, department)
VALUES (3, 'Akshay', 'akshay@gmail.com', 19, 'IT');
-- SUCCESS
```

DEFAULT fills omitted column

```
INSERT INTO constraint_demo (id, name, email_id, age)
VALUES (4, 'Sagar', 'sagar@gmail.com', 21);
-- department not listed → defaults to 'IT'

SELECT * FROM constraint_demo;
-- Sagar's row shows department = 'IT'
```

Column-Level vs Table-Level Constraints

Two ways to write the same rules

Style	Where you write it	Best for
Column-level	Right after the column type, same line	Simple single-column rules
Table-level	After all columns, at the end of `CREATE TABLE`	Named constraints, composite keys, foreign keys

They are **functionally the same** for UNIQUE / CHECK / DEFAULT / NOT NULL on one column. **PRIMARY KEY** and **FOREIGN KEY** (especially composite / named) are normally written at **table level**.

Mixed example with named constraints

```
CREATE TABLE constraint_demo2 (
    id         INT NOT NULL,
    name       VARCHAR(20),
    email_id   VARCHAR(30) UNIQUE,          -- column-level
    age        INT CHECK (age > 18),        -- column-level
    department VARCHAR(20) DEFAULT 'IT',    -- column-level
    CONSTRAINT pk_constraint_demo PRIMARY KEY (id),  -- table-level, named
    CONSTRAINT uq_name UNIQUE (name)                 -- table-level, named
);
```

Naming (`CONSTRAINT pk_constraint_demo`) is **optional but recommended**:

- Readable schema for other developers
- Easy to `DROP FOREIGN KEY name` / drop that specific constraint later

Unnamed table-level also works

```
CREATE TABLE constraint_demo2 (
    id   INT NOT NULL,
    name VARCHAR(20),
    PRIMARY KEY (id),
    UNIQUE (name)
);
```

Verify with DESCRIBE

```
DESCRIBE constraint_demo2;
```

Field	Key	Notes
`id`	`PRI`	Primary key
`name`	`UNI`	Unique (table-level)
`email_id`	`UNI`	Unique (column-level)
`age`		CHECK applied (not always visible as Key)
`department`		Default `'IT'` in Default column

ALTER TABLE: Adding and Dropping Constraints

Constraints can be added or removed **after** the table exists.

Drop and re-add primary key

```
ALTER TABLE constraint_demo2
DROP PRIMARY KEY;

DESCRIBE constraint_demo2;   -- Key column for id no longer PRI

ALTER TABLE constraint_demo2
ADD PRIMARY KEY (id);        -- restored
```

Existing data must already satisfy the new constraint

If `id` already has duplicates, adding a PK fails:

```
-- Suppose constraint_demo already has two rows with id = 4
ALTER TABLE constraint_demo
ADD PRIMARY KEY (id);
```

```
ERROR: Duplicate entry '4' for key 'constraint_demo.PRIMARY'
```

Same idea for UNIQUE / NOT NULL / CHECK: **clean the data first**, then add the constraint.

Lab experiments suggested in class

Try whether constraints can also be attached via `MODIFY` / `CHANGE`:

```
ALTER TABLE constraint_demo
MODIFY id INT NOT NULL;

ALTER TABLE constraint_demo
CHANGE id id INT NOT NULL UNIQUE;

ALTER TABLE constraint_demo
MODIFY age INT CHECK (age > 18);

ALTER TABLE constraint_demo
CHANGE department department VARCHAR(20) DEFAULT 'HR';
```

Observe what MySQL accepts; named `ADD CONSTRAINT` remains the clearest style for PKs/FKs.

Rename a column (used later in passport demo)

```
ALTER TABLE passport_application
RENAME COLUMN app_id TO app_number;
```

Data Integrity on UPDATE

Constraints apply to **UPDATE** as well as INSERT.

Duplicate PK via UPDATE — blocked

```
UPDATE constraint_demo2
SET id = 1
WHERE id = 2;
```

```
ERROR: Duplicate entry '1' for key 'PRIMARY'
```

CHECK violation via UPDATE — blocked

```sql
UPDATE constraint_demo
SET age = 15
WHERE department = 'MECH';
```

```
ERROR: Check constraint 'constraint_demo_chk_1' is violated
```

Valid UPDATE — succeeds

```sql
UPDATE constraint_demo
SET age = 25
WHERE department = 'MECH';
-- SUCCESS: 25 > 18
```

Integrity summary

Action	Blocked when...
INSERT / UPDATE	Duplicate PRIMARY KEY
INSERT / UPDATE	NULL into NOT NULL
INSERT / UPDATE	Duplicate UNIQUE value
INSERT / UPDATE	CHECK condition false
INSERT child row	FK points to missing parent (Section 9–10)

All of these protect **accuracy and consistency** of stored data.

Keys: Super, Candidate, Primary, Alternate

What is a key?

A **key** is a column (or set of columns) used to **uniquely identify** a row. Keys also support **relationships** between tables (via foreign keys).

Running example: **passport registration**

Column	Unique in real life?
Registration ID	Yes
Email ID	Yes (usually)
Mobile	Yes (usually)
Aadhaar Number	Yes
PAN Number	Yes
Name	No (many Anikets)
Gender	No
DOB	No
Address	No
Photo	Unique per person, but BLOB — not used as a key in practice

```
Super keys
   └── Candidate keys  (minimal super keys)
         ├── Primary key     (the one you choose)
         └── Alternate keys  (the other candidates)
```

Super key

Any column or combination that uniquely identifies a row — even if it contains extra unnecessary columns.

Single-column super keys: Registration ID, Email, Mobile, Aadhaar, PAN

Combination super keys (also valid):

- Registration ID + Name
- Mobile + PAN
- Aadhaar + Name

Not super keys: Name, Gender, DOB, Address (duplicates possible)

> Super key is a **design concept**. MySQL has **no** `SUPER KEY` **keyword**.

Candidate key

A **minimal** super key: unique, and **no proper subset** is still unique.

- Registration ID alone → candidate key
- Registration ID + Name → **not** a candidate key (Registration ID alone is enough — Name is redundant)

All single-column unique identifiers above are candidate keys.

> No `CANDIDATE KEY` keyword in MySQL — conceptual only.

Primary key

One candidate key you promote as the table's main identifier.

Why prefer Registration ID over Email / Aadhaar / Mobile?

Reason	Explanation
Owns the table	Registration ID exists for *this* system only
Stability	Email/mobile change; Registration ID should not
Shared elsewhere	Aadhaar/PAN appear in other systems/tables → awkward as this table's PK

Primary key properties (exam list):

- No duplicates
- No NULLs
- Only one PK per table (composite allowed)
- Creates a unique **index** → faster `SELECT` / joins
- Can be referenced by foreign keys in child tables
- **Every table should have a PK** (strongly recommended)

Alternate key

Any candidate key **not** chosen as the primary key.

If Registration ID is PK → Email, Mobile, Aadhaar, PAN are **alternate keys**.

In MySQL you often enforce alternate keys with `UNIQUE` constraints.

> No `ALTERNATE KEY` keyword — conceptual; implement with `UNIQUE`.

Composite and Surrogate Keys

Composite primary key

Used when **no single column** is unique enough. The **combination** must be unique; individuals may repeat.

```sql
CREATE TABLE student_course (
    name VARCHAR(30),
    dob  DATE,
    PRIMARY KEY (name, dob)
);
```

name	dob	Allowed?
Sahil Patil	2000-01-01	First insert OK
Aniket Takmare	2000-01-01	OK — different name, same DOB
Sahil Patil	2000-01-01	**Rejected** — same pair

Name can repeat; DOB can repeat; **(name, dob)** cannot.

> Prefer a surrogate `id` when natural composites get messy — but understand composites for exams.

Surrogate key

An **artificial** identifier with **no business meaning**, usually `INT AUTO_INCREMENT`.

```sql
CREATE TABLE passport_registration (
    id     INT AUTO_INCREMENT PRIMARY KEY,   -- surrogate
    name   VARCHAR(30),
    email  VARCHAR(50),
    -- ...
);
```

Property	Why it helps
System-generated	No user typing errors in the PK
Compact INT	Small, fast joins/indexes
Stable	User changes email → PK unchanged
Table-local	Not shared with Aadhaar/PAN systems

Why not Email as PK? A typo fix would force updates in **every child table** that references that email. With a surrogate `id`, only the email column changes.

Foreign Keys and Relationships

Relationship types

Type	Meaning	Example
One-to-One	Each A row ↔ exactly one B row	Person ↔ one passport booklet record
One-to-Many	One parent → many children	One registration → many applications
Many-to-Many	Many A ↔ many B	Authors ↔ Books (needs a **junction** table)

Day 3 focus: **one-to-many** via foreign key.

Foreign key definition

- A **foreign key (FK)** in the **child** table references a **PRIMARY KEY** (or UNIQUE key) in the **parent** table.
- Declared as a **table-level** constraint.
- FK columns **may be NULL** (unlike PK) — "no parent linked yet".
- Rule: child cannot reference a parent id that **does not exist**.



Live demo: parent + child

```sql
-- Parent
CREATE TABLE passport_registration (
    id       INT PRIMARY KEY,
    name     VARCHAR(20),
    aadhaar  INT,
    pan_number INT
);

-- Child with foreign key
CREATE TABLE passport_application (
    app_id          INT PRIMARY KEY,
    application_date DATE,
    status          VARCHAR(20),
    registration_id INT,
    FOREIGN KEY (registration_id)
        REFERENCES passport_registration(id)
);
```

Add FK after creation:

```sql
ALTER TABLE passport_application
ADD FOREIGN KEY (registration_id)
REFERENCES passport_registration(id);
```

```sql
DESCRIBE passport_application;
-- registration_id Key = MUL
-- MUL = non-unique index; MySQL marks FK columns this way
```

Sample data (one-to-many)

```sql
INSERT INTO passport_registration VALUES
    (1, 'Aniket', 123456, 1234),
    (2, 'Rohit',  456789, 5678),
    (3, 'Akshay', 128890, 1234);

INSERT INTO passport_application VALUES
    (1, '2026-09-10', 'Rejected',  1),  -- Aniket, attempt 1
    (2, '2026-10-01', 'Submitted', 1),  -- Aniket, attempt 2
    (3, '2025-05-08', 'Successful', 2); -- Rohit
-- Akshay registered but never applied → no child row
```

Join (old comma + WHERE style from class)

```sql
SELECT pr.id, pr.name, pr.aadhaar, pr.pan_number,
       pa.app_id, pa.application_date, pa.status, pa.registration_id
FROM passport_registration AS pr,
     passport_application  AS pa
WHERE pr.id = pa.registration_id;
```

Result idea	Why
Aniket × 2 rows	Two applications
Rohit × 1	One application
Akshay missing	No matching child row (inner-style join)

`WHERE pr.id = pa.registration_id` is the **join condition**. Same pattern as modern `INNER JOIN … ON …` (Day 4).

CDAC eligibility analogy (one-to-many)

```
registration (id, name, email)     ← parent
course (course_id, course_name)
eligible (eligible_id, course_name, registration_id)  ← child, FK → registration.id

Rohit (registration_id = 1) eligible for AC, BDA, AI
→ three child rows, one parent
```

Parent = table whose PK is referenced. **Child** = table that holds the FK.

Insert without valid parent — fails

```sql
-- Sagar tries to apply without registering (id 5 does not exist)
INSERT INTO passport_application (app_number, application_date, status, registration_id)
VALUES (4, '2026-10-10', 'Applied', 5);
```

```
ERROR: Cannot add or update a child row: foreign key constraint fails
```

This is **referential integrity**: every FK value must exist in the parent (or be NULL if allowed).

ON DELETE / ON UPDATE Actions

When a parent row is **deleted** or its PK is **updated**, what should happen to child rows? MySQL lets you declare the policy on the FK.

Three policies practiced

Action	On parent DELETE	On parent UPDATE	Child rows
CASCADE	Child rows **deleted**	Child FK **updated** to new PK	Removed / updated
SET NULL	Child FK → **NULL**	Child FK → **NULL**	**Preserved** (FK null)
RESTRICT	**Error** if children exist	**Error** if children exist	Unchanged

> You can mix: e.g. `ON DELETE CASCADE ON UPDATE RESTRICT`. Class practiced matching pairs.

CASCADE — recreate child with cascade

```
DROP TABLE passport_application;

CREATE TABLE passport_application (
    app_number       INT PRIMARY KEY,
    application_date DATE,
    status           VARCHAR(50),
    registration_id  INT,
    FOREIGN KEY (registration_id)
        REFERENCES passport_registration(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

DELETE CASCADE:

```
DELETE FROM passport_registration WHERE id = 1;  -- Aniket
SELECT * FROM passport_application;
-- Aniket's application rows automatically gone
```

UPDATE CASCADE:

```
UPDATE passport_registration SET id = 10 WHERE name = 'Rohit';
SELECT * FROM passport_application;
-- Rohit's registration_id automatically became 10
```

SET NULL — keep history, clear the link

FK column must allow NULL.

```
CREATE TABLE passport_application (
    app_number       INT PRIMARY KEY,
    application_date DATE,
    status           VARCHAR(50),
    registration_id  INT,
    CONSTRAINT fk_passport_app
        FOREIGN KEY (registration_id)
        REFERENCES passport_registration(id)
        ON DELETE SET NULL
        ON UPDATE SET NULL
);
```

Named constraint `fk_passport_app` makes later `DROP FOREIGN KEY` easy.

```sql
DELETE FROM passport_registration WHERE name = 'Akshay';
SELECT * FROM passport_application;
-- Akshay's apps remain; registration_id is NULL

UPDATE passport_registration SET id = 13 WHERE pan = '246';
-- referencing child registration_id becomes NULL
```

RESTRICT — block parent change while children exist

```sql
ALTER TABLE passport_application
DROP FOREIGN KEY fk_passport_app;

ALTER TABLE passport_application
ADD CONSTRAINT fk_passport_app
FOREIGN KEY (registration_id)
REFERENCES passport_registration(id)
ON DELETE RESTRICT
ON UPDATE RESTRICT;
```

```sql
DELETE FROM passport_registration WHERE name = 'Aniket';
-- ERROR: Cannot delete or update a parent row: foreign key constraint fails
-- (applications still reference Aniket)

DELETE FROM passport_registration WHERE name = 'Ritesh';
-- SUCCESS if Ritesh never applied (no child rows)
```

To delete a restricted parent: **delete (or reassign) children first**, then delete parent.

When to use which (extra guidance)

Policy	Good for
CASCADE	Dependent data that must die with the parent (e.g. cart lines when cart is deleted)
SET NULL	Keep history after parent gone (old applications without a live registration)
RESTRICT	Prevent accidents — e.g. don't delete a registration while an application is in progress

---

Aggregate Functions

Aggregates take **many rows** and return **one value** (per group, if `GROUP BY` is used).

Example table: `student (id, student_name, marks, city, course, email_id)`.

COUNT

```sql
SELECT COUNT(*) AS total_rows FROM student;
-- 7 — every row, including those with NULLs

SELECT COUNT(marks) AS marks_count FROM student;
-- counts only non-NULL marks

SELECT COUNT(student_name) AS student_count FROM student;
-- counts non-NULL names (duplicate names still count as separate rows)
```

Form	Counts
`COUNT(*)`	All rows
`COUNT(column)`	Rows where column **IS NOT NULL**
`COUNT(DISTINCT column)`	Distinct non-NULL values

SUM / AVG / MIN / MAX

```sql
SELECT SUM(marks) AS total_marks FROM student;     -- e.g. 171 (NULLs ignored)
SELECT AVG(marks) AS average_marks FROM student;   -- e.g. 29.5 (NULLs out of num & den)
SELECT MIN(marks) AS lowest_marks FROM student;    -- e.g. 27
SELECT MAX(marks) AS highest_marks FROM student;   -- e.g. 34
```

> **NULL rule:** `SUM`, `AVG`, `MIN`, `MAX`, and `COUNT(col)` ignore NULLs. Only `COUNT(*)` includes every row.

GROUP BY, HAVING, ORDER BY, DISTINCT

GROUP BY

Splits rows into **groups** that share the same value(s), then runs aggregates **per group**.

```sql
SELECT city, SUM(marks) AS total_marks
FROM student
GROUP BY city;
-- e.g. Kharghar: 59, Pune: 88, Kolhapur: 30

SELECT city, AVG(marks) AS avg_marks
FROM student
GROUP BY city;
```

Selecting non-grouped columns — error

```sql
-- WRONG: student_name neither aggregated nor in GROUP BY
SELECT city, student_name, AVG(marks)
FROM student
GROUP BY city;
```

```
ERROR: student_name is not in GROUP BY and not an aggregate function
```

Rule: In `SELECT` with `GROUP BY`, every selected column must be either:

1. Listed in `GROUP BY`, or
2. Inside an aggregate (`SUM`, `AVG`, ...)

If you `GROUP BY city, student_name` and names are unique, you effectively get one row per student — grouping becomes pointless for city totals.

HAVING — filter groups

`HAVING` filters **after** aggregation. Use it for conditions on aggregates.

```sql
SELECT city, SUM(marks) AS total_marks
FROM student
GROUP BY city
HAVING SUM(marks) > 30;
-- Kharghar, Pune

SELECT city, SUM(marks) AS total_marks
FROM student
GROUP BY city
HAVING city = 'Pune';
```

WHERE vs HAVING (memorize)

	WHERE	HAVING
Filters	Individual **rows**	**Groups** / aggregates
Aggregates in condition?	**No**	**Yes**
Timing	Before `GROUP BY`	After `GROUP BY`
Needs GROUP BY?	No	Meaningful with GROUP BY

```sql
-- Filter rows first, then group
SELECT city, AVG(marks)
FROM student
WHERE marks IS NOT NULL
GROUP BY city
HAVING AVG(marks) ≥ 29;
```

ORDER BY

```sql
SELECT * FROM student ORDER BY id ASC;           -- default ASC
SELECT * FROM student ORDER BY marks DESC;
SELECT * FROM student ORDER BY student_name;

SELECT city, SUM(marks) AS total_marks
FROM student
GROUP BY city
ORDER BY city DESC;
-- Pune, Kolhapur, Kharghar
```

DISTINCT

```sql
SELECT DISTINCT course FROM student;
-- unique courses only

SELECT DISTINCT city FROM student;

SELECT COUNT(DISTINCT student_name) AS unique_students
FROM student;
-- Gauri twice → counted once
```

Need	Use
Unique values only	`DISTINCT`
Unique values **+ totals/averages**	`GROUP BY` (+ aggregates)

LIMIT, OFFSET, BETWEEN, IN, LIKE, IS NULL

LIMIT — top N

```sql
SELECT * FROM student
ORDER BY marks DESC
LIMIT 1;            -- top scorer

SELECT * FROM student
ORDER BY marks DESC
LIMIT 5;            -- top 5
```

LIMIT + OFFSET — skip rows

`OFFSET` skips N rows; must be used **with** `LIMIT`.

```sql
-- 2nd highest
SELECT * FROM student
ORDER BY marks DESC
LIMIT 1 OFFSET 1;

-- 3rd highest
SELECT * FROM student
ORDER BY marks DESC
LIMIT 1 OFFSET 2;
```

```
ORDER BY marks DESC
Row rank:  1st (highest)  2nd  3rd  ...
LIMIT 1 OFFSET 1  → skip 1st, take next 1 → 2nd highest
```

BETWEEN — inclusive range

```sql
SELECT * FROM student
WHERE marks BETWEEN 27 AND 30;
-- equivalent to: marks ≥ 27 AND marks ≤ 30
```

IN / NOT IN

```sql
SELECT * FROM student
WHERE city IN ('Kharghar', 'Pune');

SELECT * FROM student
WHERE city NOT IN ('Pune');
```

LIKE — pattern matching

Wildcard	Matches
`%`	Any sequence of characters (including empty)
`_`	**Exactly one** character

```sql
-- 'a' anywhere
SELECT * FROM student WHERE student_name LIKE '%a%';

-- starts with A
SELECT * FROM student WHERE student_name LIKE 'A%';

-- ends with a
SELECT * FROM student WHERE student_name LIKE '%a';

-- second character is a
SELECT * FROM student WHERE student_name LIKE '_a%';

-- fifth character is a (four underscores then a)
SELECT * FROM student WHERE student_name LIKE '____a%';
```

Pattern	Meaning
`'A%'`	Starts with A
`'%a'`	Ends with a
`'%a%'`	Contains a
`'_a%'`	2nd char is a
`'____a%'`	5th char is a

LIKE is case-(in)sensitive depending on collation; for class practice, treat patterns carefully with your data's casing.

IS NULL / IS NOT NULL

```sql
-- WRONG — always empty
SELECT * FROM student WHERE email_id = NULL;

-- RIGHT
SELECT * FROM student WHERE email_id IS NULL;
SELECT * FROM student WHERE email_id IS NOT NULL;
```

Logical Query Execution Order

You **write** SQL roughly as `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`.
The engine **executes** in a different order:

Step	Clause	What happens
1	**FROM**	Identify source table(s)
2	**WHERE**	Filter individual rows
3	**GROUP BY**	Group remaining rows
4	**HAVING**	Filter groups
5	**SELECT**	Choose columns / compute aggregates
6	**ORDER BY**	Sort result
7	**LIMIT / OFFSET**	Restrict how many rows returned

Why this matters

- `WHERE` **cannot use** `SUM(marks) > 30` — aggregates don't exist yet at step 2. Use `HAVING`.
- Aliases in `SELECT` are often **not** usable in `WHERE` (WHERE ran earlier).
- `OFFSET` needs `LIMIT` declared with it.
- `ORDER BY` can use aliases from `SELECT` in MySQL because sorting is after select-list evaluation.

```
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
```

Revision Checklist and Next Steps

Constraints

- NOT NULL, UNIQUE, PRIMARY KEY, DEFAULT, CHECK
- Column-level vs table-level; named `CONSTRAINT` for clean drops
- UNIQUE allows multiple NULLs; PK does not allow NULL

Keys

- Super → Candidate → Primary / Alternate
- Composite PK; Surrogate (`AUTO_INCREMENT`) preferred for stability

Relationships and FK actions

- Parent / child; one-to-many pattern (passport, CDAC eligibility)
- Referential integrity: child cannot point to missing parent
- **CASCADE / SET NULL / RESTRICT** on DELETE and UPDATE

Aggregates and clauses

- `COUNT(*)` vs `COUNT(col)` vs `COUNT(DISTINCT col)`
- `SUM`, `AVG`, `MIN`, `MAX` ignore NULL
- `GROUP BY`, `HAVING` vs `WHERE`, `ORDER BY`, `DISTINCT`
- `LIMIT` / `OFFSET`, `BETWEEN`, `IN`, `LIKE`, `IS NULL`
- Logical execution order

ALTER TABLE (Day 2 + 3)

- ADD / MODIFY / CHANGE / RENAME COLUMN / DROP COLUMN
- DROP PRIMARY KEY / ADD PRIMARY KEY
- DROP FOREIGN KEY `name` / ADD CONSTRAINT … FOREIGN KEY

Practice tasks from class

1. Experiment adding constraints with `MODIFY` / `CHANGE`.
2. Rebuild passport tables and practice all six ON DELETE/UPDATE behaviors.
3. Write your own `GROUP BY` + `HAVING` + `ORDER BY` queries on `student`.

Coming next

- **JOINs** (INNER, LEFT, RIGHT) — major follow-on to FKs
- Set ops (UNION, …) may appear later
- Window functions mentioned as a future topic (per-row aggregates beside group stats)

> Coverage: DBT Day 3 Parts 1 & 2 (10 Oct 2026) — constraints, key theory, FK referential actions, aggregates and filtering — expanded with explanations and runnable MySQL examples beyond the raw PDF.